

Collection Framework

The Java Collection Framework (JCF) is a set of interfaces, classes, and utility methods that provides ready-made data structures for storing and manipulating groups of objects.

Instead of building our own data structures, Java provides implementations such as `ArrayList`, `LinkedList`, `HashSet`, `HashMap`, and `PriorityQueue`.

Important: `Map` is part of the Collection Framework, but it does not extend the `Collection` interface.

A reading order:

What it is → The hierarchy → The List classes → The Set classes
→ The Queue classes → The Map classes → Legacy and concurrent
→ Choosing one

1. What it is

Three parts:

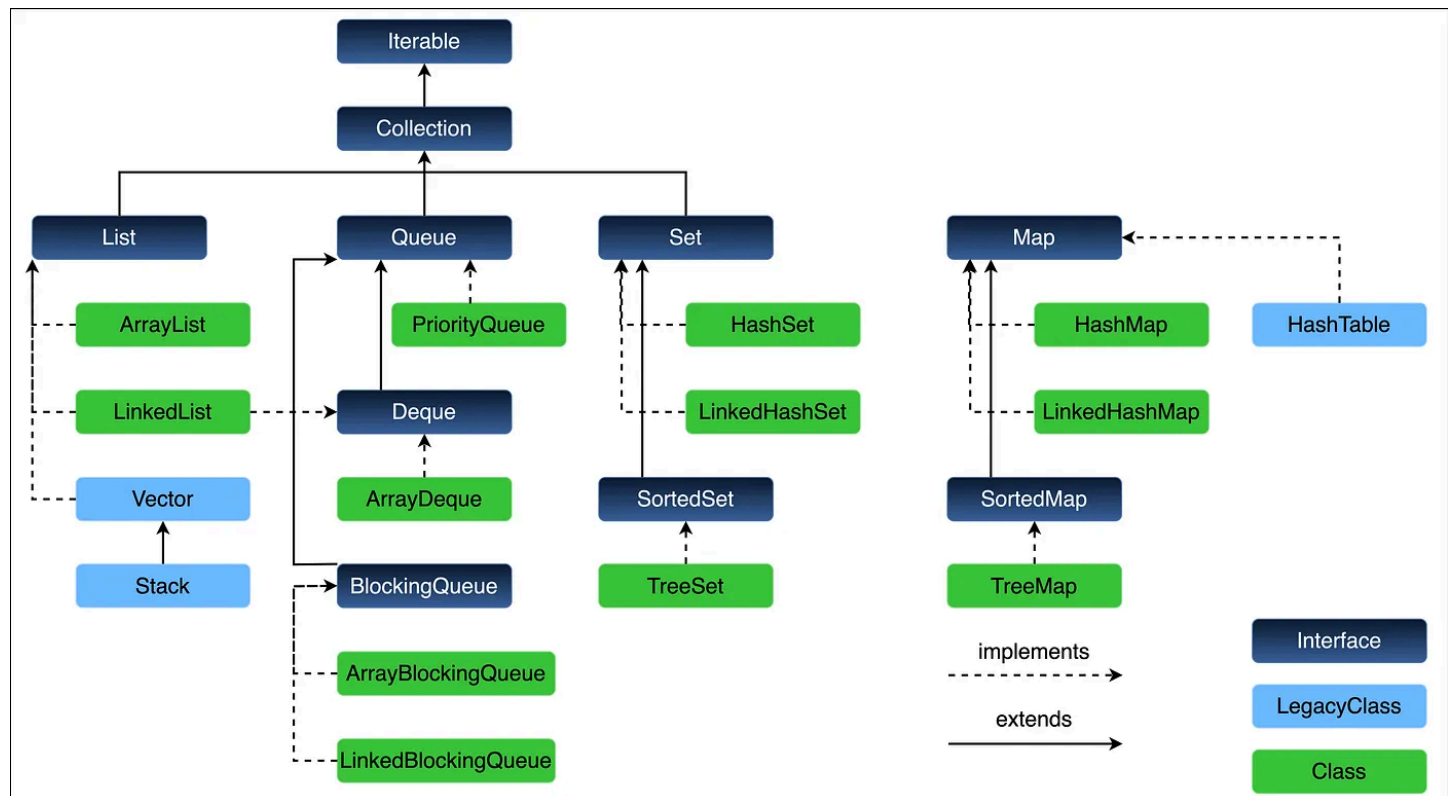
Part	Examples
Interfaces	<code>List</code> , <code>Set</code> , <code>Queue</code> , <code>Map</code>
Implementations	<code>ArrayList</code> , <code>HashSet</code> , <code>ArrayDeque</code> , <code>HashMap</code>
Helpers	<code>Collections.sort</code> , <code>Arrays.asList</code> , <code>List.of</code>

Declare the interface, create the implementation:

```
List<String> names = new ArrayList<>();
```

Changing `ArrayList` to `LinkedList` is then one word, and no caller notices. Declaring `ArrayList` names instead ties every method that receives it to that one class.

2. The hierarchy



An arrow points from a type to the one above it. Solid is extends, dashed is implements.

Interface	What it promises
Collection	A group of elements: add, remove, size, iterate
List	Order, and access by index. Duplicates allowed
Set	No duplicates
Queue	Elements waiting their turn, in a defined order
Deque	A queue that is open at both ends
SortedSet , SortedMap	The same, kept in order
BlockingQueue	A queue where taking from it can wait
Map	Keys mapped to values. No duplicate keys

Three things in that picture are worth naming.

LinkedList appears under two parents. It implements **List** and **Deque** , which is why it can be used as a list, a queue or a stack.

The blue boxes are older than the framework. They predate it and were retrofitted, which is why they sit slightly apart on the diagram. Section 13 covers what to use instead.

The diagram simplifies two of the lines. `LinkedHashSet` extends `HashSet` rather than implementing `Set` directly, and `LinkedHashMap` extends `HashMap`. Both add ordering to an existing implementation rather than starting again.

3. ArrayList

`ArrayList` is a resizable array that stores elements in insertion order and allows duplicates.

Reading by index is immediate. Inserting in the middle shifts everything after it.

```
public class Main {
    public static void main(String[] args) {
        List<String> names = new ArrayList<>();

        names.add("Ravi");
        names.add("Amit");
        names.add("Ravi");           // duplicates are allowed

        names.add(1, "Priya");       // insert at an index

        System.out.println(names);
        System.out.println(names.get(2));
        System.out.println(names.indexOf("Ravi"));
        System.out.println(names.contains("Neha"));
    }
}
```

Output:

```
[Ravi, Priya, Amit, Ravi]
Amit
0
false
```

Use it unless there is a reason not to. It is the right answer for almost every list, and the exceptions are narrow.

4. LinkedList

`LinkedList` is a list where each element is connected to the next element.

Adding at either end is immediate. Reaching index 500 means following 500 links.

```

public class Main {
    public static void main(String[] args) {
        LinkedList<String> queue = new LinkedList<>();

        queue.add("Amit");
        queue.addFirst("Ravi");           // only a LinkedList or Deque can do this
        queue.addLast("Priya");

        System.out.println(queue);
        System.out.println(queue.getFirst());
        System.out.println(queue.removeFirst());
        System.out.println(queue);
    }
}

```

Output:

```

[Ravi, Amit, Priya]
Ravi
Ravi
[Amit, Priya]

```

`LinkedList` implements `List` and `Deque`, which is why it has `addFirst` and `removeFirst` as well as `get(index)`.

It is chosen more often than it should be. For work at the ends, `ArrayDeque` is faster and uses less memory. For work by index, `ArrayList` wins. That leaves very little.

5. HashSet

`HashSet` stores unique elements without maintaining insertion order.

```

public class Main {
    public static void main(String[] args) {
        Set<String> names = new HashSet<>();

        System.out.println(names.add("Ravi"));
        System.out.println(names.add("Amit"));
        System.out.println(names.add("Ravi"));    // already present

        System.out.println(names.size());
        System.out.println(names.contains("Amit"));
    }
}

```

Output:

```
true
true
false
2
true
```

`add` returns whether the set changed, which is a cheap way to test whether a value has been seen before.

It decides duplicates with `equals` and `hashCode`. A class that defines neither is compared by identity, so two objects with identical fields are both stored.

6. LinkedHashSet

`LinkedHashSet` stores unique elements while maintaining insertion order.

```
public class Main {
    public static void main(String[] args) {
        Set<String> names = new LinkedHashSet<>();

        names.add("Ravi");
        names.add("Amit");
        names.add("Priya");
        names.add("Amit");

        System.out.println(names);
    }
}
```

Output:

```
[Ravi, Amit, Priya]
```

Use it when the output is read by a person. A `HashSet` would print the same three names in an order that can change between runs, which makes a report or a test hard to trust.

7. TreeSet

`TreeSet` stores unique elements in sorted order.

```

public class Main {
    public static void main(String[] args) {
        TreeSet<String> names = new TreeSet<>(List.of("Ravi", "Amit", "Priya"));

        System.out.println(names);
        System.out.println(names.first());
        System.out.println(names.last());
        System.out.println(names.headSet("Priya"));
        System.out.println(names.ceiling("B"));
    }
}

```

Output:

```

[Amit, Priya, Ravi]
Amit
Ravi
[Amit]
Priya

```

Sorting costs something on every insert, and in exchange the range methods come free: `first`, `last`, `headSet`, `tailSet`, `ceiling`, `floor`.

It cannot hold `null`, because null cannot be compared.

8. ArrayDeque

`ArrayDeque` is a resizable double-ended queue that allows adding and removing elements from both ends.

It is the right choice for a queue and for a stack.

```

public class Main {
    public static void main(String[] args) {
        Deque<String> queue = new ArrayDeque<>();

        queue.offer("first");
        queue.offer("second");
        System.out.println(queue.poll());    // first in, first out

        Deque<String> stack = new ArrayDeque<>();
        stack.push("a");
        stack.push("b");
        System.out.println(stack.pop());    // last in, first out
    }
}

```

Output:

```
first
b
```

Each operation has two forms, and they differ only in how they fail:

Task	Throws when it cannot	Returns a value instead
Insert	<code>add</code>	<code>offer</code>
Remove	<code>remove</code>	<code>poll</code>
Look	<code>element</code>	<code>peek</code>

Use it instead of `Stack`, which extends `Vector`, is synchronized whether or not that helps, and lets a caller reach into the middle.

9. PriorityQueue

`PriorityQueue` is a queue that processes elements based on their priority.

```
public class Main {
    public static void main(String[] args) {
        PriorityQueue<Integer> queue = new PriorityQueue<>();

        queue.offer(30);
        queue.offer(10);
        queue.offer(20);

        System.out.println(queue.poll());
        System.out.println(queue.poll());
        System.out.println(queue.poll());
    }
}
```

Output:

```
10
20
30
```

A `Comparator` reverses it or orders objects:

```
PriorityQueue<Order> byValue =  
    new PriorityQueue<>(Comparator.comparing(Order::total).reversed());
```

Only the head is ordered. Printing the queue itself shows the internal heap, not a sorted list, so a loop of `poll` is the only way to read it in order.

10. HashMap

`HashMap` stores data as key-value pairs and allows fast lookup using a key.

It is the default map, and it keeps no order that can be relied on.

```
public class Main {  
    public static void main(String[] args) {  
        Map<String, Integer> scores = new HashMap<>();  
  
        scores.put("Ravi", 90);  
        scores.put("Amit", 85);  
        scores.put("Ravi", 95);           // replaces, does not add  
  
        System.out.println(scores.get("Ravi"));  
        System.out.println(scores.size());  
        System.out.println(scores.getOrDefault("Neha", 0));  
  
        for (Map.Entry<String, Integer> entry : scores.entrySet()) {  
            System.out.println(entry.getKey() + " = " + entry.getValue());  
        }  
    }  
}
```

Output, with the two entries in whichever order the map holds them:

```
95  
2  
0  
Ravi = 95  
Amit = 85
```

Four methods remove most of the `if` statements around a map:

Method	What it does
<code>getOrDefault(k, d)</code>	The value, or <code>d</code> when the key is absent
<code>putIfAbsent(k, v)</code>	Stores only when the key is absent
<code>computeIfAbsent(k, f)</code>	Stores the result of <code>f</code> when the key is absent
<code>merge(k, v, f)</code>	Combines a new value with any existing one

```
counts.merge(word, 1, Integer::sum);           // counting, with no null check
```

It allows one null key and any number of null values.

11. LinkedHashMap

`LinkedHashMap` is a `Map` that stores key-value pairs and maintains insertion order.

```
public class Main {
    public static void main(String[] args) {
        Map<String, Integer> scores = new LinkedHashMap<>();

        scores.put("Ravi", 90);
        scores.put("Amit", 85);
        scores.put("Priya", 78);

        System.out.println(scores);
    }
}
```

Output:

```
{Ravi=90, Amit=85, Priya=78}
```

Access order turns it into a cache that forgets the least recently used entry:

```
new LinkedHashMap<>(16, 0.75f, true) {
    @Override
    protected boolean removeEldestEntry(Map.Entry<String, Integer> eldest) {
        return size() > 100;
    }
};
```

Use it when the order of the output matters, or when the access-order form gives a cache for nothing. A `HashMap` is cheaper when neither does.

12. TreeMap

`TreeMap` is a `Map` that stores key-value pairs in sorted order of keys.

```
public class Main {
    public static void main(String[] args) {
        TreeMap<String, Integer> scores = new TreeMap<>();

        scores.put("Ravi", 90);
        scores.put("Amit", 85);
        scores.put("Priya", 78);

        System.out.println(scores);
        System.out.println(scores.firstKey());
        System.out.println(scores.headMap("Priya"));
    }
}
```

Output:

```
{Amit=85, Priya=78, Ravi=90}
Amit
{Amit=85}
```

Use it when a range or an order is part of the question: every entry before a date, the lowest key, a report that must come out alphabetically. Otherwise a `HashMap` is cheaper.

Like `TreeSet`, it rejects a null key.

13. The legacy classes

`Vector`, `Stack` and `Hashtable` are from Java 1.0 and were fitted into the framework afterwards.

Legacy	Replace with
<code>Vector</code>	<code>ArrayList</code>
<code>Stack</code>	<code>ArrayDeque</code>
<code>Hashtable</code>	<code>HashMap</code> , or <code>ConcurrentHashMap</code>

All three synchronize every method, which costs something on every call and still leaves a sequence of calls unsafe.

14. Choosing one

The first question is what shape the data has:

```
keys and values      → Map
no duplicates        → Set
order and an index   → List
waiting their turn   → Queue or Deque
```

Then one more question inside each:

If also	List	Set	Map
Nothing special	<code>ArrayList</code>	<code>HashSet</code>	<code>HashMap</code>
Insertion order matters	<code>ArrayList</code>	<code>LinkedHashSet</code>	<code>LinkedHashMap</code>
Sorted	Sort the <code>ArrayList</code>	<code>TreeSet</code>	<code>TreeMap</code>
Several threads	<code>CopyOnWriteArrayList</code>	<code>ConcurrentHashMap.newKeySet()</code>	<code>ConcurrentHashMap</code>

Start with `ArrayList`, `HashSet` and `HashMap`. They are right far more often than not, and the ordered and sorted versions charge something for a guarantee most code does not need.

Two rules apply whichever one is chosen.

Removing while iterating throws.

```
for (String name : names) {
    if (name.equals("Amit")) {
        names.remove(name);           // ConcurrentModificationException
    }
}

names.removeIf(name -> name.equals("Amit")); // correct, and shorter
```

`List.of` and friends cannot be changed.

```
List<String> fixed = List.of("Ravi", "Amit");
fixed.add("Priya");           // UnsupportedOperationException
```

That makes them safe to share and safe to return from a method, and it is why they reject nulls.

The pieces together

Class	Order	Choose it when
<code>ArrayList</code>	Insertion	The default list
<code>LinkedList</code>	Insertion	Rarely. <code>ArrayDeque</code> or <code>ArrayList</code> usually wins
<code>HashSet</code>	None	The default set
<code>LinkedHashSet</code>	Insertion	The output is read by a person
<code>TreeSet</code>	Sorted	Ranges, or a sorted report
<code>ArrayDeque</code>	Insertion	A queue or a stack
<code>PriorityQueue</code>	By priority	The smallest or largest comes next
<code>HashMap</code>	None	The default map
<code>LinkedHashMap</code>	Insertion or access	Predictable output, or a cache
<code>TreeMap</code>	Sorted by key	Ranges, or a sorted report
<code>ConcurrentHashMap</code>	None	Several threads

Take an order service. The lines on an order are an `ArrayList`, because order matters and the same product can appear twice. The countries shipped to are a `HashSet`. Orders waiting to be processed sit in an `ArrayDeque`, and the ones being retried in a `PriorityQueue` ordered by attempt count. Prices by product id are a `HashMap`, filled with `computeIfAbsent`. The monthly report uses a `TreeMap`, because it has to come out in order, and nothing else does.

Take one collection in a real project and name the promise it actually needs. Most `LinkedList` declarations turn out to want an `ArrayList`.